

PR_10.1 Dani Gayol Rodríguez

PR_10.1 Dani Gayol Rodríguez.....	1
Apartado A.....	1
1.) Crea un clúster en MongoAtlas, carga los datos de ejemplo sample_training y:	2
a.) Adjunta capturas de pantalla del:	2
i.) Dashboard del clúster	2
ii.) Bases de datos / colecciones creadas	2
b.) Conéctate mediante MongoDB Compass y adjunta una captura de pantalla tras conectar con el clúster.....	4
c.) En la base de datos drfleming, crea un documento con tus datos personales en la colección alumnos. Tras insertarlo, realiza una consulta para recuperarlo.	5
d.) Recupera las colecciones existentes en la base de datos.	7
Apartado B.....	8
1.) Haciendo uso de mongosh, escribe los comandos necesarios y adjunta el resultado de las consultas para:	9
a.) Obtener las bases de datos creadas.	9
b.) Sobre la base de datos sample_training	9
i.) Recuperar todas las colecciones.....	9
ii.) Recuperar qué base de datos está activa.	9
c.) Sobre la colección sample_training.zips:	9
i.) Recuperar todos los documentos.....	10
ii.) Recuperar el primer documento.	10
iii.) Recuperar los documentos de la ciudad de SAN DIEGO.	11
iv.) Recuperar los códigos postales (zip) cuya ciudad sea SAN DIEGO y el estado sea CA.	11
v.) Recuperar los documentos cuya ciudad sea SAN DIEGO pero no pertenezcan al estado de CA.	12
vi.) Recuperar los documentos cuya ciudad sea SAN DIEGO y el estado sea CA o TX.	12

vii.) Recuperar los documentos cuya ciudad sea SAN DIEGO o el estado sea TX.....	13
viii.) Recuperar los documentos donde el zip sea 92135, 92136 o 92137.	14
ix.) ¿Cuál es el código postal de la ciudad de ALLEN que no tiene habitantes (sólo recupera el zip, no nos interesa ningún otro campo, ni el _id)?	15
2.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección zips de la base de datos sample_training:	15
a.) Recupera los documentos que tienen menos de 50 personas (campo pop). ...	16
b.) Recupera los documentos que tienen entre 50 y 100 personas.	16
c.) Averigua cuantos documentos hay de la ciudad de SAN DIEGO.	17
d.) Averigua cuantos documentos tienen menos de 100 personas.	17
e.) Recupera los documentos donde el nombre de la ciudad contenga la palabra BEACH.	17
f.) Obtén los estados de la ciudad de SAN DIEGO (Solución: ['CA', 'TX'])......	18
g.) Listado con los 5 códigos postales más poblados (muestra los documentos completos).....	18
h.) Cantidad de documentos que no tienen menos de 5.000 habitantes ni más de 1.000.000 (debes utilizar el operador \$nor).	19
i.) Cuantos documentos tienen más habitantes que su propio código postal (campo zip).	20
Apartado C	21
1.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección trips de la base de datos sample_training:	22
a.) Recupera los viajes realizados con la bicicleta con ID 14785	22
b.) Encuentra los viajes que duraron más de 1 hora.	22
c.) Mostrar solo el ID de la bicicleta, la duración del viaje y los nombres de las estaciones de los viajes realizados por usuarios nacidos después de 1990.	23
d.) Recupera los viajes realizados desde la estación Huron St & Franklin St y con duración menor a 500 segundos	24
e.) Encontrar los 10 viajes más largos, ordenados de mayor a menor duración. ...	25
f.) Encontrar viajes con estaciones que contengan Ave en su nombre.	26
g.) ¿Cuántos viajes han realizado los usuarios de tipo Customer?.	27
h.) Recupera los viajes realizados entre el 1 de enero y el 31 de marzo de 2016.	27

i.) Encontrar viajes con duración entre 10 y 20 minutos, hechos por usuarios nacidos antes de 1980.	28
j.) Encontrar los viajes que duraron más de una hora o que fueron realizados por usuarios nacidos después del año 2000.	29
k.) Mostrar los 5 viajes más recientes, pero saltando los primeros 10 resultados.	30
2.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección sales de la base de datos sample_supplies:.....	31
a.) Recupera los documentos que han vendido algún producto (forma parte del array items) de nombre notepad.	32
b.) Recupera los documentos donde el primer producto sea notepad.	33
c.) Recupera los documentos donde el primer producto sea notepad y haya vendido un mínimo de 5 (quantity) unidades.	34
d.) Recupera los documentos que han vendido exactamente dos productos.	35
e.) Averiguar las ventas y sus productos donde el precio de los productos es superior a 20€ pero inferior a 40€ (muestra solo los productos que tengan dichos precios).....	36
f.) Recupera las ventas donde sus productos tengan las etiquetas stationary, office y general.	37
g.) Recupera las ventas donde sus productos tengan las etiquetas writing, office o electronics.	38
h.) Para las ventas que sean notepad o laptop, recuperar el primer producto.	39
i.) Recupera las ventas realizadas por hombres de más de 40 años que hayan comprado dos productos.	40
j.) Para las ventas realizadas por mujeres y realizadas por teléfono (purchaseMethod), recupera dos de los productos adquiridos.	41
Apartado D	42
1.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección posts:	43
a.) Recupera los mensajes que tienen la etiqueta club.....	43
b.) Recupera los mensajes que tienen la etiqueta club y spy.	43
c.) Recupera los mensajes que tienen la etiqueta club o spy.	44
d.) Cuantos mensajes no tienen la etiqueta restaurant.	45
e.) Cuantos mensajes tienen únicamente un comentario.....	45
f.) Cuantos mensajes tienen al menos un comentario.....	45

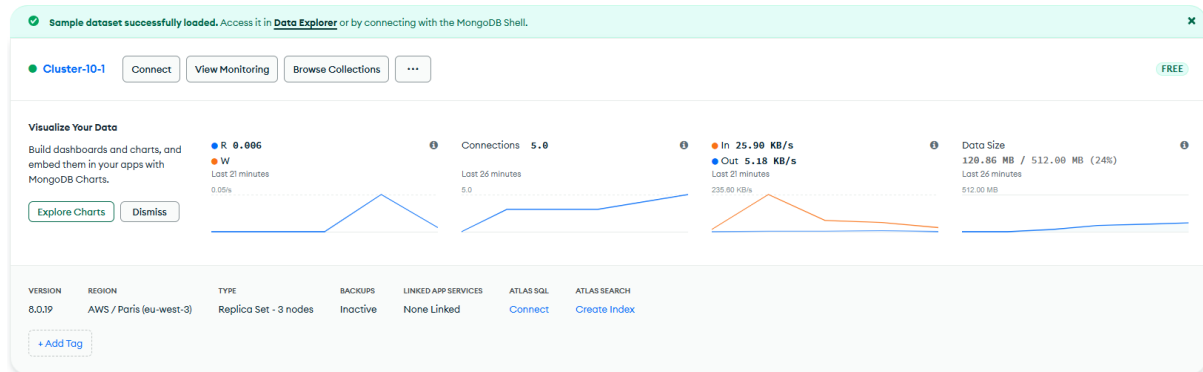
g.) Los mensajes que tienen algún comentario que ha escrito el usuario Salena Olmos.....	46
h.) Los comentarios (con su cuerpo y email) que ha escrito el usuario Salena Olmos.....	46
i.) Recupera los mensajes que en body contengan la palabra earth, y devuelve el título, 3 comentarios y 5 etiquetas.....	47

Apartado A

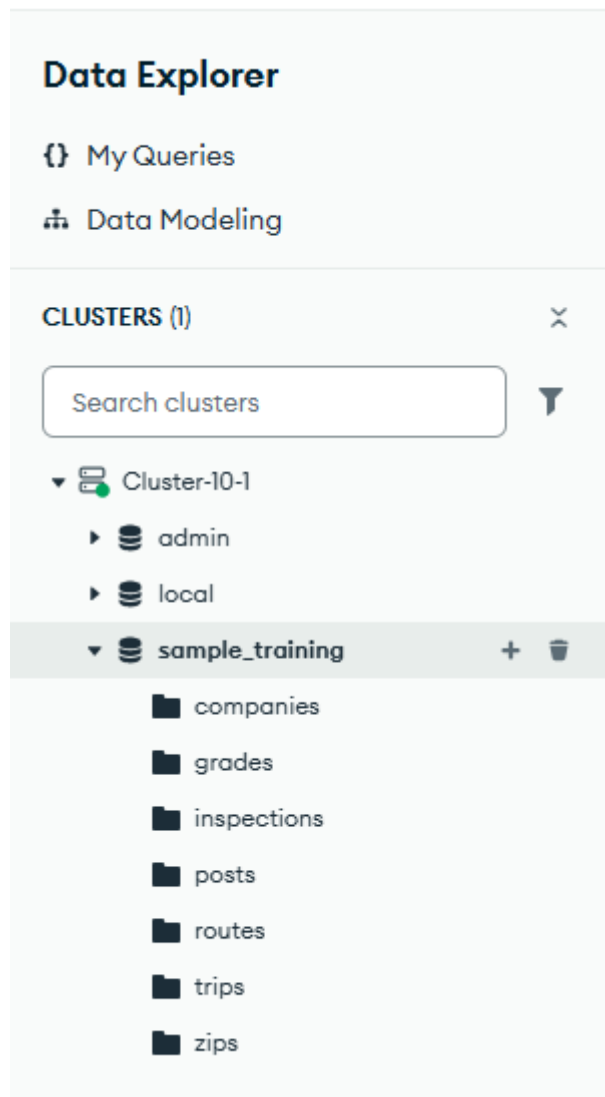
1.) Crea un clúster en MongoDB Atlas, carga los datos de ejemplo `sample_training` y:

a.) Adjunta capturas de pantalla del:

i.) Dashboard del clúster



ii.) Bases de datos / colecciones creadas



Cluster-10-1 > sample_training > companies

Documents 9.5K

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

UPDATE

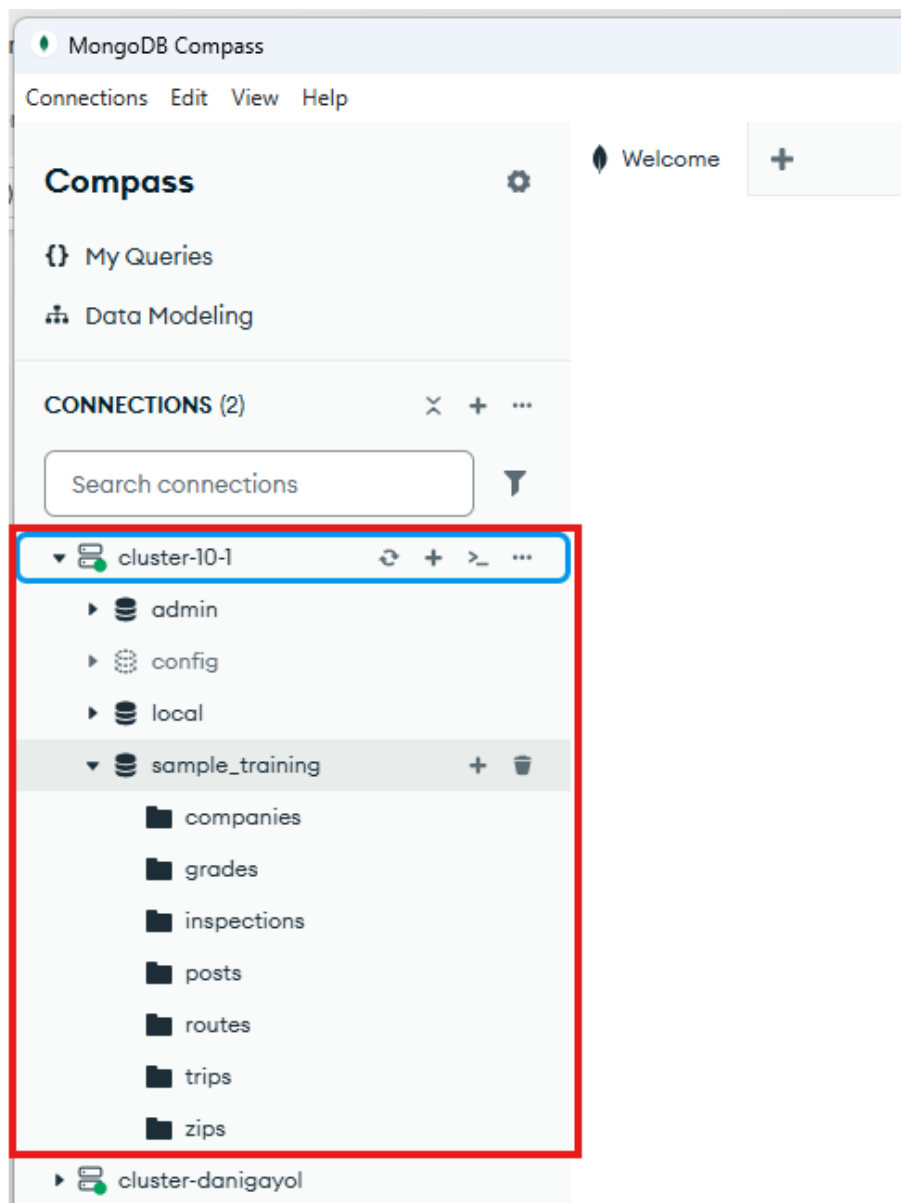
DELETE

EXPORT CODE

```
_id: ObjectId('52cdef7c4bab8bd675297d8a')
name: "Wetpaint"
permalink: "abc2"
crunchbase_url: "http://www.crunchbase.com/company/wetpaint"
homepage_url: "http://wetpaint-inc.com"
blog_url: "http://digitalquarters.net/"
blog_feed_url: "http://digitalquarters.net/feed/"
twitter_username: "BachelrWetpaint"
category_code: "web"
number_of_employees: 47
founded_year: 2005
founded_month: 10
founded_day: 17
deadpooled_year: 1
tag_list: "wiki, seattle, elowitz, media-industry, media-platform, social-distrib..."
alias_list: ""
email_address: "info@wetpaint.com"
phone_number: "206.859.6300"
description: "Technology Platform Company"
created_at: 2007-05-25T06:51:27.000+00:00
updated_at: "Sun Dec 08 07:15:44 UTC 2013"
overview: "<p>Wetpaint is a technology platform company that uses its proprietary..."
  image: Object
  products: Array (2)
  relationships: Array (17)
  Show 13 more fields
```

b.) Conéctate mediante MongoDB Compass y adjunta una captura de pantalla tras conectar con el clúster.

IP Address	Comment	Status	Actions
158.99.18.30/32 (includes your current IP address)	Created as part of the Auto Setup process	Active	EDIT DELETE
0.0.0.0/0 (includes your current IP address)		Active	EDIT DELETE



c.) En la base de datos drfleming, crea un documento con tus datos personales en la colección alumnos. Tras insertarlo, realiza una consulta para recuperarlo.

Create Database

Database Name

drfleming

Collection Name

alumnos

☐ Time-Series

Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

> Additional preferences

(e.g. Custom collation, Clustered collections)

Cancel

Create Database

Insert Document

To collection drfleming.alumnos

VIEW

{ }

≡

1 /**

2 * Paste one or more documents here

3 */

4 {

5 "nombre": "Dani",

6 "apellido": "Gayol Rodríguez",

7 "edad": 2,

8 "email": "ydr29913@educastur.es",

9 "curso": "IA y Big Data"

10 }

Cancel

Insert

cluster-10-1 > drfleming > alumnos

Open MongoDB shell

Documents 1

Aggregations

Schema

Indexes 1

Validation

{nombre: "Dani"}

Generate query

Explain

Reset

Find

Options

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

25

1 - 1 of 1

≡

{ }

⌂

_id: ObjectId('699597d691660afec029f23c')

nombre: "Dani"

apellido: "Gayol Rodríguez"

edad: 2

email: "ydr29913@educastur.es"

curso: "IA y Big Data"

d.) Recupera las colecciones existentes en la base de datos.

cluster-10-1 > drfleming

Open MongoDB shell

Create collection

Refresh

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
alumnos	-	4.10 kB	1	138.00 B	1	4.10 kB

Apartado B

1.) Haciendo uso de mongosh, escribe los comandos necesarios y adjunta el resultado de las consultas para:

a.) Obtener las bases de datos creadas.

```
>_MONGOSH
> show dbs
< drfleming      72.00 KiB
  sample_training 65.56 MiB
  admin          360.00 KiB
  local          4.07 GiB
```

b.) Sobre la base de datos sample_training

```
> use sample_training
< switched to db sample_training
```

i.) Recuperar todas las colecciones

```
> show collections
< companies
  grades
  inspections
  posts
  routes
  trips
  zips
```

ii.) Recuperar qué base de datos está activa.

```
> db
< sample_training
```

c.) Sobre la colección sample_training.zips:

i.) Recuperar todos los documentos.

```
> db.zips.find()
< {
  _id: ObjectId('5c8eccc1caa187d17ca6ed16'),
  city: 'ALPINE',
  zip: '35014',
  loc: {
    y: 33.331165,
    x: 86.208934
  },
  pop: 3062,
  state: 'AL'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6ed17'),
  city: 'BESSEMER',
  zip: '35020',
  loc: {
    y: 33.409002,
    x: 86.947547
  },
  pop: 40549,
  state: 'AL'
}
```

ii.) Recuperar el primer documento.

```
> db.zips.findOne()
< {
  _id: ObjectId('5c8eccc1caa187d17ca6ed16'),
  city: 'ALPINE',
  zip: '35014',
  loc: {
    y: 33.331165,
    x: 86.208934
  },
  pop: 3062,
  state: 'AL'
}
```

iii.) Recuperar los documentos de la ciudad de SAN DIEGO.

```
> db.zips.find({ city: "SAN DIEGO" })
< {
  _id: ObjectId('5c8eccc1caa187d17ca6f4b5'),
  city: 'SAN DIEGO',
  zip: '92101',
  loc: {
    y: 32.71852,
    x: 117.159316
  },
  pop: 20265,
  state: 'CA'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6f4b7'),
  city: 'SAN DIEGO',
  zip: '92102',
  loc: {
    y: 32.713893,
    x: 117.121858
  },
  pop: 45265,
  state: 'CA'
}
```

iv.) Recuperar los códigos postales (zip) cuya ciudad sea SAN DIEGO y el estado sea CA.

```
> db.zips.find(
  { city: "SAN DIEGO", state: "CA" },
  { zip: 1, _id: 0 }
)
< {
  zip: '92101'
}
{
  zip: '92102'
}
{
  zip: '92103'
}
{
  zip: '92104'
}
{
  zip: '92105'
}
```

v.) Recuperar los documentos cuya ciudad sea SAN DIEGO pero no pertenezcan al estado de CA.

```
> db.zips.find({
  city: "SAN DIEGO",
  state: { $ne: "CA" }
})
< {
  _id: ObjectId('5c8eccc1caa187d17ca75196'),
  city: 'SAN DIEGO',
  zip: '78384',
  loc: {
    y: 27.76535,
    x: 98.250297
  },
  pop: 5018,
  state: 'TX'
}
```

vi.) Recuperar los documentos cuya ciudad sea SAN DIEGO y el estado sea CA o TX.

```
> db.zips.find({
  city: "SAN DIEGO",
  state: { $in: ["CA", "TX"] }
})
< {
  _id: ObjectId('5c8eccc1caa187d17ca6f4b5'),
  city: 'SAN DIEGO',
  zip: '92101',
  loc: {
    y: 32.71852,
    x: 117.159316
  },
  pop: 20265,
  state: 'CA'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6f4b7'),
  city: 'SAN DIEGO',
  zip: '92102',
  loc: {
    y: 32.713893,
    x: 117.121858
  },
  pop: 45265,
  state: 'CA'
}
```


vii.) Recuperar los documentos cuya ciudad sea SAN DIEGO o el estado sea TX.

```
> db.zips.find({
  $or: [
    { city: "SAN DIEGO" },
    { state: "TX" }
  ]
})
< {
  _id: ObjectId('5c8eccc1caa187d17ca6f4b5'),
  city: 'SAN DIEGO',
  zip: '92101',
  loc: {
    y: 32.71852,
    x: 117.159316
  },
  pop: 20265,
  state: 'CA'
}
```

viii.) Recuperar los documentos donde el zip sea 92135, 92136 o 92137.

```
> db.zips.find({
  zip: { $in: ["92135", "92136", "92137"] }
})
< {
  _id: ObjectId('5c8eccc1caa187d17ca6f4d7'),
  city: 'SAN DIEGO',
  zip: '92136',
  loc: {
    y: 32.681585,
    x: 117.124678
  },
  pop: 11750,
  state: 'CA'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6f4e9'),
  city: 'SAN DIEGO',
  zip: '92135',
  loc: {
    y: 32.702482,
    x: 117.19202
  },
  pop: 8122,
  state: 'CA'
}
```

ix.) ¿Cuál es el código postal de la ciudad de ALLEN que no tiene habitantes (sólo recupera el zip, no nos interesa ningún otro campo, ni el _id)?

```
> db.zips.find(
  { city: "ALLEN", pop: 0 },
  { zip: 1, _id: 0 }
)
< {
  zip: '36419'
}
```

2.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección zips de la base de datos sample_training:

a.) Recupera los documentos que tienen menos de 50 personas (campo pop).

```
> db.zips.find({
  pop: { $lt: 50 }
})
< {
  _id: ObjectId('5c8eccc1caal87d17ca6eea3'),
  city: 'ALLEN',
  zip: '36419',
  loc: {
    y: 31.624266,
    x: 87.66746
  },
  pop: 0,
  state: 'AL'
}
{
  _id: ObjectId('5c8eccc1caal87d17ca6eebf'),
  city: 'CARLTON',
  zip: '36515',
  loc: {
    y: 31.322449,
    x: 87.837793
  },
  pop: 30,
  state: 'AL'
}
```

b.) Recupera los documentos que tienen entre 50 y 100 personas.

```
> db.zips.find({
  pop: { $gte: 50, $lte: 100 }
})
< {
  _id: ObjectId('5c8eccc1caa187d17ca6ef67'),
  city: 'CLARKS POINT',
  zip: '99569',
  loc: {
    y: 58.84921,
    x: 158.451241
  },
  pop: 68,
  state: 'AK'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6ef6c'),
  city: 'CHENEGA BAY',
  zip: '99574',
  loc: {
    y: 60.102558,
    x: 147.943316
  },
  pop: 96,
  state: 'AK'
}
```

c.) Averigua cuantos documentos hay de la ciudad de SAN DIEGO.

```
> db.zips.countDocuments({ city: "SAN DIEGO" })
< 35
```

d.) Averigua cuantos documentos tienen menos de 100 personas.

```
> db.zips.countDocuments({
  pop: { $lt: 100 }
})
< 716
```

e.) Recupera los documentos donde el nombre de la ciudad contenga la palabra BEACH.

```
> db.zips.find({
  city: { $regex: "BEACH" }
})
< {
  _id: ObjectId('5c8eccc1caa187d17ca6eee2'),
  city: 'ORANGE BEACH',
  zip: '36561',
  loc: {
    y: 30.290596,
    x: 87.555888
  },
  pop: 2549,
  state: 'AL'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6f3ac'),
  city: 'HERMOSA BEACH',
  zip: '90254',
  loc: {
    y: 33.864309,
    x: 118.395511
  },
  pop: 18289,
  state: 'CA'
}
```

f.) Obtén los estados de la ciudad de SAN DIEGO (Solución: ['CA', 'TX']).

```
> db.zips.distinct("state", { city: "SAN DIEGO" })
< [ 'CA', 'TX' ]
```

g.) Listado con los 5 códigos postales más poblados (muestra los documentos completos).

```
> db.zips.find()
  .sort({ pop: -1 })
  .limit(5)
< {
  _id: ObjectId('5c8eccc1caa187d17ca7044d'),
  city: 'CHICAGO',
  zip: '60623',
  loc: {
    y: 41.849015,
    x: 87.7157
  },
  pop: 112047,
  state: 'IL'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca7307f'),
  city: 'BROOKLYN',
  zip: '11226',
  loc: {
    y: 40.646694,
    x: 73.956985
  },
  pop: 111396,
  state: 'NY'
}
```

h.) Cantidad de documentos que no tienen menos de 5.000 habitantes ni más de 1.000.000 (debes utilizar el operador \$nor).

```
> db.zips.find({
  $nor: [
    { pop: { $lt: 5000 } },
    { pop: { $gt: 1000000 } }
  ]
})
< {
  _id: ObjectId('5c8eccc1caa187d17ca6ed17'),
  city: 'BESSEMER',
  zip: '35020',
  loc: {
    y: 33.409002,
    x: 86.947547
  },
  pop: 40549,
  state: 'AL'
}
{
  _id: ObjectId('5c8eccc1caa187d17ca6ed18'),
  city: 'ACMAR',
  zip: '35004',
  loc: {
    y: 33.584132,
    x: 86.51557
  },
  pop: 6055,
  state: 'AL'
}
```

i.) Cuantos documentos tienen más habitantes que su propio código postal (campo zip).

```
> db.zips.countDocuments({
  $expr: {
    $gt: [
      "$pop",
      { $toInt: "$zip" }
    ]
  }
})
< 2332
```


Apartado C

1.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección trips de la base de datos sample_training:

a.) Recupera los viajes realizados con la bicicleta con ID 14785

```
> db.trips.find({ bikeid: 14785 })
< {
  _id: ObjectId('572bb8222b288919b68abf82'),
  tripduration: 812,
  'start station id': 518,
  'start station name': 'E 39 St & 2 Ave',
  'end station id': 433,
  'end station name': 'E 13 St & Avenue A',
  bikeid: 14785,
  usertype: 'Subscriber',
  'birth year': 1977,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.9734419,
      40.74780373
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.98057249,
      40.72955361
    ]
  },
  'start time': 2016-01-01T00:14:48.000Z,
  'stop time': 2016-01-01T00:28:21.000Z
}
```

b.) Encuentra los viajes que duraron más de 1 hora.

```
> db.trips.find({ tripduration: { $gt: 3600 } })
< {
  _id: ObjectId('572bb8222b288919b68ac07c'),
  tripduration: 326222,
  'start station id': 391,
  'start station name': 'Clark St & Henry St',
  'end station id': 310,
  'end station name': 'State St & Smith St',
  bikeid: 18591,
  usertype: 'Subscriber',
  'birth year': 1979,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.99344559,
      40.69760127
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.98912867,
      40.68926942
    ]
  },
  'start time': 2016-01-01T00:58:20.000Z,
  'stop time': 2016-01-04T19:35:23.000Z
}
```

c.) Mostrar solo el ID de la bicicleta, la duración del viaje y los nombres de las estaciones de los viajes realizados por usuarios nacidos después de 1990.

```
> db.trips.find(
  { "birth year": { $gt: 1990 } },
  {
    _id: 0,
    bikeid: 1,
    tripduration: 1,
    "start station name": 1,
    "end station name": 1
  }
)
< {
  tripduration: 694,
  'start station name': 'Howard St & Centre St',
  'end station name': 'E 17 St & Broadway',
  bikeid: 15747
}
{
  tripduration: 479,
  'start station name': 'Franklin St & Dupont St',
  'end station name': 'Kent Ave & N 7 St',
  bikeid: 23563
}
{
  tripduration: 428,
  'start station name': '6 Ave & W 33 St',
  'end station name': 'W 15 St & 7 Ave',
  bikeid: 23176
}
```

d.) Recupera los viajes realizados desde la estación Huron St & Franklin St y con duración menor a 500 segundos

```
> db.trips.find({
  "start station name": "Huron St & Franklin St",
  tripduration: { $lt: 500 }
})
< {
  _id: ObjectId('572bb8232b288919b68af1c4'),
  tripduration: 413,
  'start station id': 3116,
  'start station name': 'Huron St & Franklin St',
  'end station id': 3104,
  'end station name': 'Kent Ave & N 7 St',
  bikeid: 16757,
  usertype: 'Subscriber',
  'birth year': 1978,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.95826,
      40.73266
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.96150225,
      40.72057658
    ]
  },
  'start time': 2016-01-02T10:18:16.000Z,
  'stop time': 2016-01-02T10:25:09.000Z
}
```

e.) Encontrar los 10 viajes más largos, ordenados de mayor a menor duración.

```
> db.trips.find()
.sort({ tripduration: -1 })
.limit(10)
< {
  _id: ObjectId('572bb8222b288919b68ac07c'),
  tripduration: 326222,
  'start station id': 391,
  'start station name': 'Clark St & Henry St',
  'end station id': 310,
  'end station name': 'State St & Smith St',
  bikeid: 18591,
  usertype: 'Subscriber',
  'birth year': 1979,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.99344559,
      40.69760127
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.98912867,
      40.68926942
    ]
  },
  'start time': 2016-01-01T00:58:20.000Z,
  'stop time': 2016-01-04T19:35:23.000Z
}
```

f.) Encontrar viajes con estaciones que contengan Ave en su nombre.

```
> db.trips.find({
  $or: [
    { "start station name": { $regex: "Ave" } },
    { "end station name": { $regex: "Ave" } }
  ]
})
< {
  _id: ObjectId('572bb822b288919b68abf5a'),
  tripduration: 379,
  'start station id': 476,
  'start station name': 'E 31 St & 3 Ave',
  'end station id': 498,
  'end station name': 'Broadway & W 32 St',
  bikeid: 17827,
  usertype: 'Subscriber',
  'birth year': 1969,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.97966069,
      40.74394314
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.98808416,
      40.74854862
    ]
  },
  'start time': 2016-01-01T00:00:45.000Z,
  'stop time': 2016-01-01T00:07:04.000Z
}
```

g.) ¿Cuántos viajes han realizado los usuarios de tipo Customer?.

```
> db.trips.countDocuments({ usertype: "Customer" })
< 1989
```

h.) Recuper los viajes realizados entre el 1 de enero y el 31 de marzo de 2016.

```
> db.trips.find({
  "start time": {
    $gte: ISODate("2016-01-01T00:00:00Z"),
    $lte: ISODate("2016-03-31T23:59:59Z")
  }
})
< {
  _id: ObjectId('572bb8222b288919b68abf5a'),
  tripduration: 379,
  'start station id': 476,
  'start station name': 'E 31 St & 3 Ave',
  'end station id': 498,
  'end station name': 'Broadway & W 32 St',
  bikeid: 17827,
  usertype: 'Subscriber',
  'birth year': 1969,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.97966069,
      40.74394314
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.98808416,
      40.74854862
    ]
  },
  'start time': 2016-01-01T00:00:45.000Z,
  'stop time': 2016-01-01T00:07:04.000Z
}
```

i.) Encontrar viajes con duración entre 10 y 20 minutos, hechos por usuarios nacidos antes de 1980.

```
> db.trips.find({
  tripduration: { $gte: 600, $lte: 1200 },
  "birth year": { $lt: 1980 }
})
< {
  _id: ObjectId('572bb8222b288919b68abf5b'),
  tripduration: 889,
  'start station id': 268,
  'start station name': 'Howard St & Centre St',
  'end station id': 3002,
  'end station name': 'South End Ave & Liberty St',
  bikeid: 22794,
  usertype: 'Subscriber',
  'birth year': 1961,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.99973337,
      40.71910537
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -74.015756,
      40.711512
    ]
  },
  'start time': 2016-01-01T00:01:06.000Z,
  'stop time': 2016-01-01T00:15:56.000Z
}
```


j.) Encontrar los viajes que duraron más de una hora o que fueron realizados por usuarios nacidos después del año 2000.

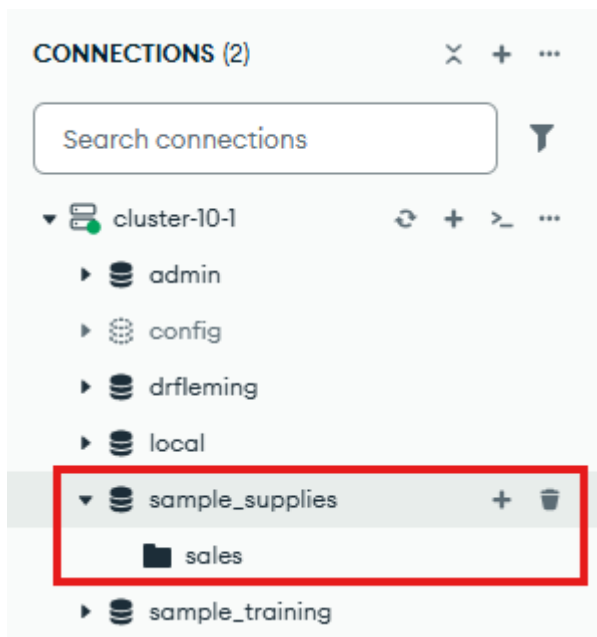
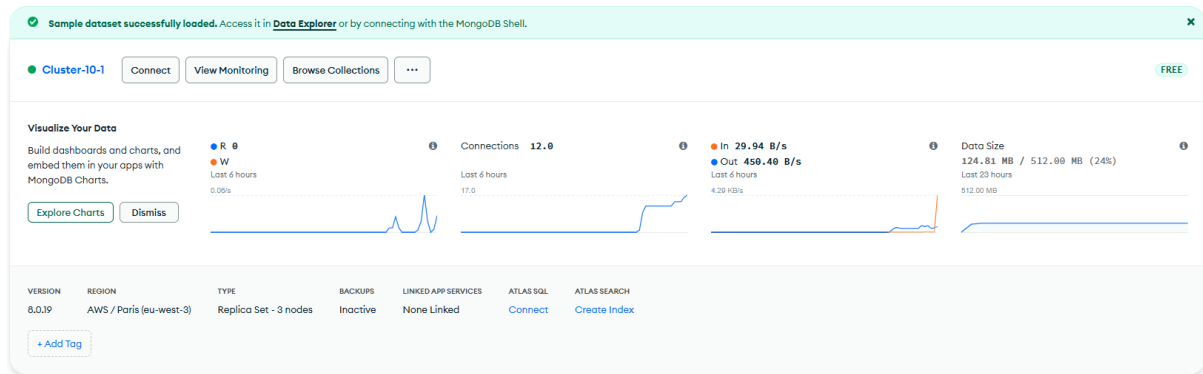
```
> db.trips.find({
  $or: [
    { tripduration: { $gt: 3600 } },
    { "birth year": { $gt: 2000 } }
  ]
})
< {
  _id: ObjectId('572bb8222b288919b68ac07c'),
  tripduration: 326222,
  'start station id': 391,
  'start station name': 'Clark St & Henry St',
  'end station id': 310,
  'end station name': 'State St & Smith St',
  bikeid: 18591,
  usertype: 'Subscriber',
  'birth year': 1979,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.99344559,
      40.69760127
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.98912867,
      40.68926942
    ]
  },
  'start time': 2016-01-01T00:58:20.000Z,
  'stop time': 2016-01-04T19:35:23.000Z
}
```

k.) Mostrar los 5 viajes más recientes, pero saltando los primeros 10 resultados.

```
> db.trips.find()
.sort({ starttime: -1 })
.skip(10)
.limit(5)
< {
  _id: ObjectId('572bb8222b288919b68abf5a'),
  tripduration: 379,
  'start station id': 476,
  'start station name': 'E 31 St & 3 Ave',
  'end station id': 498,
  'end station name': 'Broadway & W 32 St',
  bikeid: 17827,
  usertype: 'Subscriber',
  'birth year': 1969,
  'start station location': {
    type: 'Point',
    coordinates: [
      -73.97966069,
      40.74394314
    ]
  },
  'end station location': {
    type: 'Point',
    coordinates: [
      -73.98808416,
      40.74854862
    ]
  },
  'start time': 2016-01-01T00:00:45.000Z,
  'stop time': 2016-01-01T00:07:04.000Z
}
```

2.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección sales de la base de datos sample_supplies:

Como no tenía cargada la base de datos de “sample_supplies”, voy a cargarla ahora:



```
> use sample_supplies
< switched to db sample_supplies
```

a.) Recupera los documentos que han vendido algún producto (forma parte del array items) de nombre notepad.

```
> db.sales.find({
  "items.name": "notepad"
})
< {
  _id: ObjectId('5bd761dcae323e45a93ccfe8'),
  saleDate: 2015-03-23T21:06:49.506Z,
  items: [
    {
      name: 'printer paper',
      tags: [
        'office',
        'stationary'
      ],
      price: Decimal128('40.01'),
      quantity: 2
    },
    {
      name: 'notepad',
      tags: [
        'office',
        'writing',
        'school'
      ],
      price: Decimal128('35.29'),
      quantity: 2
    },
  ],
}
```

b.) Recupera los documentos donde el primer producto sea notepad.

```
> db.sales.find({
  "items.0.name": "notepad"
})
< {
  _id: ObjectId('5bd761dcae323e45a93ccfea'),
  saleDate: 2017-06-22T09:54:14.185Z,
  items: [
    {
      name: 'notepad',
      tags: [
        'office',
        'writing',
        'school'
      ],
      price: Decimal128('30.08'),
      quantity: 1
    },
    {
      name: 'binder',
      tags: [
        'school',
        'general',
        'organization'
      ],
      price: Decimal128('24.68'),
      quantity: 7
    },
  ],
}
```

c.) Recupera los documentos donde el primer producto sea notepad y haya vendido un mínimo de 5 (quantity) unidades.

```
> db.sales.find({
  "items.0.name": "notepad",
  "items.0.quantity": { $gte: 5 }
})
< {
  _id: ObjectId('5bd761dcae323e45a93ccffe'),
  saleDate: 2013-03-20T16:36:39.112Z,
  items: [
    {
      name: 'notepad',
      tags: [
        'office',
        'writing',
        'school'
      ],
      price: Decimal128('25.83'),
      quantity: 5
    },
    {
      name: 'printer paper',
      tags: [
        'office',
        'stationary'
      ],
      price: Decimal128('14.6'),
      quantity: 2
    },
  ],
}
```

d.) Recupera los documentos que han vendido exactamente dos productos.

```
> db.sales.find({
  items: { $size: 2 }
})
< {
  _id: ObjectId('5bd761dcae323e45a93ccfed'),
  saleDate: 2015-09-02T16:11:59.565Z,
  items: [
    {
      name: 'binder',
      tags: [
        'school',
        'general',
        'organization'
      ],
      price: Decimal128('13.44'),
      quantity: 8
    },
    {
      name: 'binder',
      tags: [
        'school',
        'general',
        'organization'
      ],
      price: Decimal128('16.66'),
      quantity: 10
    }
  ],
  storeLocation: 'London',
  customer: {
    gender: 'M',
    age: 44,
    email: 'owtar@pu.cd',
    satisfaction: 2
  }
}
```

e.) Averiguar las ventas y sus productos donde el precio de los productos es superior a 20€ pero inferior a 40€ (muestra solo los productos que tengan dichos precios).

```
> db.sales.aggregate([
  {
    $project: {
      _id: 0,
      saleDate: 1,
      items: {
        $filter: {
          input: "$items",
          as: "item",
          cond: {
            $and: [
              { $gt: ["$$item.price", 20] },
              { $lt: ["$$item.price", 40] }
            ]
          }
        }
      }
    }
  }
])
< {
  saleDate: 2015-03-23T21:06:49.506Z,
  items: [
    {
      name: 'notepad',
      tags: [
        'office',
        'writing',
        'school'
      ],
      price: Decimal128('35.29'),
      quantity: 2
    }
  ]
}
```


f.) Recupera las ventas donde sus productos tengan las etiquetas stationary, office y general.

```
> db.sales.find({
  "items.tags": {
    $all: ["stationary", "office", "general"]
  }
})
< {
  _id: ObjectId('5bd761dcae323e45a93ccfe8'),
  saleDate: 2015-03-23T21:06:49.506Z,
  items: [
    {
      name: 'printer paper',
      tags: [
        'office',
        'stationary'
      ],
      price: Decimal128('40.01'),
      quantity: 2
    },
    {
      name: 'notepad',
      tags: [
        'office',
        'writing',
        'school'
      ],
      price: Decimal128('35.29'),
      quantity: 2
    },
  ],
}
```

g.) Recupera las ventas donde sus productos tengan las etiquetas writing, office o electronics.

```
> db.sales.find({
  "items.tags": {
    $in: ["writing", "office", "electronics"]
  }
})
< {
  _id: ObjectId('5bd761dcae323e45a93ccfe8'),
  saleDate: 2015-03-23T21:06:49.506Z,
  items: [
    {
      name: 'printer paper',
      tags: [
        'office',
        'stationary'
      ],
      price: Decimal128('40.01'),
      quantity: 2
    },
    {
      name: 'notepad',
      tags: [
        'office',
        'writing',
        'school'
      ],
      price: Decimal128('35.29'),
      quantity: 2
    },
    {
      name: 'pens',
      tags: [
        'writing',
        'office',
        'school',
        'stationary'
      ],
      quantity: 2
    }
  ]
}
```

h.) Para las ventas que sean notepad o laptop, recuperar el primer producto.

```
> db.sales.find(
  {
    "items.name": { $in: ["notepad", "laptop"] }
  },
  {
    _id: 0,
    "items": 1
  }
)
< {
  items: [
    {
      name: 'printer paper',
      tags: [
        'office',
        'stationary'
      ],
      price: Decimal128('40.01'),
      quantity: 2
    },
```

i.) Recupera las ventas realizadas por hombres de más de 40 años que hayan comprado dos productos.

```
> db.sales.find({
  "customer.gender": "M",
  "customer.age": { $gt: 40 },
  items: { $size: 2 }
})
< {
  _id: ObjectId('5bd761dcae323e45a93ccfed'),
  saleDate: 2015-09-02T16:11:59.565Z,
  items: [
    {
      name: 'binder',
      tags: [
        'school',
        'general',
        'organization'
      ],
      price: Decimal128('13.44'),
      quantity: 8
    },
    {
      name: 'binder',
      tags: [
        'school',
        'general',
        'organization'
      ],
      price: Decimal128('16.66'),
      quantity: 10
    }
  ],
}
```

j.) Para las ventas realizadas por mujeres y realizadas por teléfono (purchaseMethod), recupera dos de los productos adquiridos.

```
> db.sales.find(
  {
    "customer.gender": "F",
    purchaseMethod: "Phone"
  },
  {
    _id: 0,
    items: { $slice: ["$items", 2] }
  }
)
< {
  items: [
    {
      name: 'pens',
      tags: [
        'writing',
        'office',
        'school',
        'stationary'
      ],
      price: Decimal128('75.21'),
      quantity: 1
    },
    {
      name: 'backpack',
      tags: [
        'school',
        'travel',
        'kids'
      ],
      price: Decimal128('61.11'),
      quantity: 5
    }
  ]
}
```

Apartado D

1.) Escribe los comandos necesarios para realizar las siguientes consultas sobre la colección posts:

a.) Recupera los mensajes que tienen la etiqueta club.

```
> db.posts.find({
  tags: "club"
})
< {
  _id: ObjectId('50ab0f8bbcf1bfe2536dc44a'),
  body: 'Four score and seven years ago our fathers brought forth on this continent a
    '\n',
  permalink: 'BP1VYnQkbWapGLEeYXMc',
  author: 'machine',
  title: 'Gettysburg Address',
  tags: [
    'alibi',
    'boundary',
    'push',
    'club',
    'forehead',
    'lycra',
    'lisa',
    'wash',
    'hardhat',
    'spy'
  ],
  comments: [
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      email: 'QKwaughB@QkrLWrkS.com',
      author: 'Cassi Heal'
    },
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      email: 'ZaYMfKcN@RhPLMqQD.com',
      author: 'Quincy Danaher'
    },
  ],
}
```

b.) Recupera los mensajes que tienen la etiqueta club y spy.

```
> db.posts.find({
  tags: { $all: ["club", "spy"] }
})
< {
  _id: ObjectId('50ab0f8bbcf1bfe2536dc44a'),
  body: 'Four score and seven years ago our fathers brought forth on t
    '\n',
  permalink: 'BPlVYnQkbWapgL EeYXMc',
  author: 'machine',
  title: 'Gettysburg Address',
  tags: [
    'alibi',
    'boundary',
    'push',
    'club',
    'forehead',
    'lycra',
    'lisa',
    'wash',
    'hardhat',
    'spy'
  ],
  comments: [
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit,
      email: 'QKwaughB@QkrlWrkS.com',
      author: 'Cassi Heal'
    },
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit,
      email: 'ZaYMfKcN@RhplMqQD.com',
      author: 'Quincy Danaher'
    },
  ],
}
```

c.) Recupera los mensajes que tienen la etiqueta club o spy.

```
> db.posts.find({
  tags: { $in: ["club", "spy"] }
})
< {
  _id: ObjectId('50ab0f8bbcf1bfe2536dc44a'),
  body: 'Four score and seven years ago our fathers brought forth on this d
    '\n',
  permalink: 'BPlVYnQkbWapgLEeYXMc',
  author: 'machine',
  title: 'Gettysburg Address',
  tags: [
    'alibi',
    'boundary',
    'push',
    'club',
    'forehead',
    'lycra',
    'lisa',
    'wash',
    'hardhat',
    'spy'
  ],
  comments: [
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed
      email: 'QKwaughB@QkrLWrkS.com',
      author: 'Cassi Heal'
    },
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed
      email: 'ZaYMfKcN@RhplMqQD.com',
      author: 'Quincy Danaher'
    },
  ],
}
```

d.) Cuantos mensajes no tienen la etiqueta restaurant.

```
> db.posts.countDocuments({
  tags: { $ne: "restaurant" }
})
< 496
```

e.) Cuantos mensajes tienen únicamente un comentario.

```
> db.posts.countDocuments({
  comments: { $size: 1 }
})
< 0
```


f.) Cuantos mensajes tienen al menos un comentario.

```
> db.posts.countDocuments({
  comments: { $exists: true, $not: { $size: 0 } }
})
< 500
```

g.) Los mensajes que tienen algún comentario que ha escrito el usuario Salena Olmos.

```
> db.posts.find({
  "comments.author": "Salena Olmos"
})
< {
  _id: ObjectId('50ab0f8bbcf1bfe2536dc403'),
  body: 'We the People of the United States, in Order to form a more p
    <p>Article. I.<p><p>Section. 1.<p><p>All legislative Powers herei
  permalink: 'mzRnhUaDUqVmFZZaPPUy',
  author: 'machine',
  title: 'US Constitution',
  tags: [
    'chauffeur',
    'dresser',
    'religion',
    'salt',
    'dahlia',
    'umbrella',
    'grain',
    'bathroom',
    'fire',
    'shade'
  ],
  comments: [
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit,
      email: 'WPZuEfGf@WAjYsKaC.com',
      author: 'Toshiko Sabella'
    },
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit,
      email: 'LmFLKycm@hpLYgkKo.com',
      author: 'Maren Scheider'
    },
  ],
}
```

h.) Los comentarios (con su cuerpo y email) que ha escrito el usuario Salena Olmos.

```
> db.posts.aggregate([
  {
    $project: {
      _id: 0,
      comments: {
        $filter: {
          input: "$comments",
          as: "comment",
          cond: { $eq: ["$$comment.author", "Salena Olmos"] }
        }
      }
    }
  },
  {
    $unwind: "$comments"
  },
  {
    $project: {
      body: "$comments.body",
      email: "$comments.email"
    }
  }
])
< {
  body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.'
  email: 'HnQxeqCR@tXlLvAvh.com'
}
{
  body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.'
  email: 'lEqcTIRd@zUXAdWNn.com'
}
{
  body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.'
  email: 'qvnaTpaL@iclXrlmk.com'
}
```

i.) Recupera los mensajes que en body contengan la palabra earth, y devuelve el título, 3 comentarios y 5 etiquetas.

```
> db.posts.find(
  {
    body: { $regex: "earth", $options: "i" }
  },
  {
    _id: 0,
    title: 1,
    comments: { $slice: 3 },
    tags: { $slice: 5 }
  }
)
< {
  title: 'Gettysburg Address',
  tags: [
    'avenue',
    'distance',
    'copper',
    'interactive',
    'lunchroom'
  ],
  comments: [
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      email: 'qElUSapd@grcMSgIz.com',
      author: 'Linnie Weigel'
    },
    {
      body: 'Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
      email: 'ACgmDzvbgXkdkhRJ.com',
      author: 'Sadie Jernigan'
    },
  ],
}
```